Efficient Message Passing Algorithm and Architecture Co-Design for Graph Neural Networks

Xiaofeng Zou, Cen Chen, *Senior Member, IEEE*, Luochuan Zhang, Shengyang Li, Joey Tianyi Zhou, *Senior Member, IEEE*, Wei Wei, *Senior Member, IEEE*, and Kenli Li, *Senior Member, IEEE*

***Abstract*—Graph neural networks (GNNs) are a promising method for learning graph representations and demonstrate remarkable performance on various graph-related tasks. Existing typical GNNs exploit the neighborhood message passing scheme that subtly aggregates feature messages from neighbor nodes to update the node representations. Despite the effectiveness of this scheme, its complex computational model heavily relies on the graph structure, which hinders their scaling to realistic large-scale graph applications. Although several custom accelerators have been proposed to speed up GNNs, these hardware-specific optimization techniques fail to address the fundamental problem of high computational complexity in GNNs. To tackle this challenge, we propose a dedicated algorithm-architecture co-design framework, dubbed MePa, which aims to improve GNN execution efficiency by coordinating algorithm- and hardware-level innovations. Specifically, with an in-depth analysis of GNN message-passing algorithms and potential speedup opportunities, we first propose an efficient message-passing algorithm that can dynamically prune task-irrelevant graph data at multiple granularity, including channel/edge/node-wise. This approach significantly reduces the overall complexity of GNN with negligible accuracy loss. A novel architecture is designed to support dynamic pruning and translate it into performance improvements. Elaborate pipelines and specialized optimizations jointly improve performance and decrease energy consumption. Compared to the state-of-the-art GNN accelerator AWB-GCN, MePa achieves on average 1.95× speedups and 2.6× energy efficiency.**

Manuscript received 30 June 2023; revised 24 December 2023 and 2 May 2024; accepted 25 May 2024. Date of publication 8 July 2024; date of current version 23 January 2025. This work was supported by the Fundamental Research Funds for the Central Universities under Grant 2023ZYGXZR023, in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515010220, in part by the Postdoctoral Fellowship Program of CPSF under Grant GZC20230841, in part by the Basic research of Shenzhen Science and Technology Plan under Grant JCYJ20210324123802006, Sponsored by CCF-Phytium Fund, and Sponsored by CAAI-MindSpore Open Fund, developed on OpenI Community. *(Corresponding author: Cen Chen.)*

Xiaofeng Zou, Luochuan Zhang, and Shengyang Li are with the School of Future Technology, South China University of Technology, Guangzhou, Guangdong 510641, China (e-mail: zouxiaofeng@hnu.edu.cn; zlc_0704@126.com; 202130131658@mail.scut.edu.cn).

Cen Chen is with the School of Future Technology, South China University of Technology, Guangzhou, Guangdong 510641, China, also with the Shenzhen Institute, Hunan University, Changsha 518055, China, and also with the Pazhou Laboratory, Guangzhou 510330, China (e-mail: chencen@scut.edu.cn).

Joey Tianyi Zhou is with the Institute of High Performance Computing, Singapore 138632 (e-mail: joey.tianyi.zhou@gmail.com).

Wei Wei is with the School of Computer Science and Engineering, Xi'an University of Technology, Xi'an 710049, China (e-mail: weiwei@xaut.edu.cn).

Kenli Li is with the College of Information Science and Engineering, Hunan University, Changsha, Hunan 410082, China (e-mail: lkl@hnu.edu.cn).

Recommended for acceptance by A. Lam.

Digital Object Identifier 10.1109/TETCI.2024.3420692

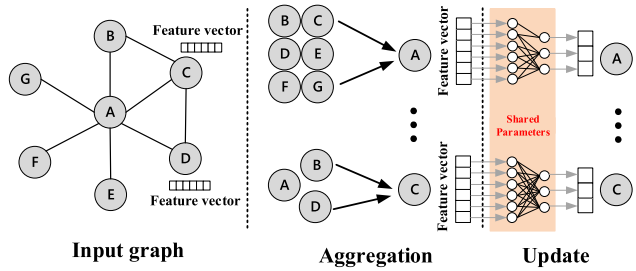


Fig. 1. Computing abstraction of neighborhood message-passing mechanism.

***Index Terms*—Algorithm and architecture co-design, graph neural networks, dynamic pruning.**

I. Introduction

RECENTLY, graph neural networks (GNNs) [1], [2], [3] have been effectively applied to numerous graph-based tasks, including recommendation systems [4] and natural language processing [5], due to their capability to accurately learn graph-structured data representations. Typical GNNs basically follow a recursive message-passing scheme, which updates the node representation by aggregating the feature messages from neighboring nodes [1]. After multiple message-passing iterations, the node representation can capture information from multi-hop neighbors. The final features are used for downstream prediction tasks, such as node classification [6], graph classification [7], community detection [8], and link prediction [9]. In brief, GNNs are becoming tools for solving complex graph networks in a broad range of application scenarios.

Despite their promising performance, GNN inference suffers from notorious inefficiency and hinders GNN scaling to realistic large-scale graph applications. The main obstacles arise from two aspects: i) Complexity and irregularity of graph data [1], [10]. Real-world graphs tend to contain a large number of nodes and edges, and follow a power-law distribution with highly irregular adjacency matrices. These unique properties result in difficult graph data processing on edge devices with limited hardware resources. ii) Hybrid computing paradigm of GNNs [11], [12]. As shown in Fig. 1, GNNs comprise two primary execution phases: aggregation and update. During the aggregation phase, feature information is collected from neighbors based on the graph structure, and the update phase transforms the aggregated features to generate new node features. The computational model of the update phase resembles that of traditional neural

networks, while the aggregation phase relies heavily on irregular and sparse graph structures, maintaining extensive graph processing behaviors. This hybrid computing paradigm involves both regular and extensive irregular computations, leading to inefficient execution.

Due to the mentioned challenges, there are several recent efforts to design GNN-specific accelerators [11], [12], [13], [14]. HyGCN [11] focused on addressing the irregularities in the aggregation and the reusability in the update by exploring intra/inter-vertex parallelism. ENGN [12] proposed the Ring-Edge-Reduce (RER) dataflow, which enables the processing of arbitrary dimensional graphs and improves accelerator scalability for large graphs. AWB-GCN [14] discovered the issue of workload imbalance in the aggregation phase and proposed an automatic adjustment algorithm to balance the workload. Although these accelerators achieve superior performance and cheaper energy consumption than GPUs for GNNs, these hardware-specific optimization techniques fail to address the fundamental problem of the high computational complexity in GNNs.

In this study, we first conduct an intensive analysis of the execution flow for GNNs. Our analysis reveals extensive redundant message passing in GNNs, which can be categorized into three main types: (i) Channel redundancy. Node feature vectors often have high dimensions, containing e numerous less important channels. (ii) Edge redundancy. Not all neighbors of a node contribute task-relevant feature messages, and the information collected from neighbors may be useful information or interfering noise. (iii) Node Redundancy. Different nodes may require varying levels of messaging. Some simple nodes converge and produce correct predictions in the early layers. Further message passing for aggregated nodes would introduce redundant computation. Although some algorithms and accelerators have leveraged pruning to improve the efficiency of GNNs, including random dropping [15], [16] or policy-based pruning [17], [18], [19], [20], these approaches primarily focus on pruning the graph structure (nodes or edges) from a single perspective, lacking a unified framework to comprehensively identify and prune multiple redundant graph data. This may be because the hybrid execution flow of GNN involves several complex operations, making it a non-trivial task to eliminate multiple redundancies.

To this end, we propose MePa, an algorithm-architecture co-design solution, which aims to efficiently identify and prune task-irrelevant graph data to accelerate GNN execution. We first propose an efficient message-passing algorithm that can dynamically prune task-irrelevant graph data at multiple granularity levels: node-wise, edge-wise, and channel-wise, significantly reducing GNN inference cost while providing comparable or even better accuracy. Furthermore, we propose a learnable threshold pruning method for efficient runtime pruning. Different from top-k pruning and predefined threshold pruning, the proposed learnable threshold pruning utilizes the gradient-based back-propagation algorithm to simultaneously optimize the weight parameters and thresholds, without relying on limited empirical evidence.

Although the proposed algorithm effectively reduces redundant message-passing, existing GNN accelerators [11], [12], [13], [14] can not support such fine-grained multiple pruning. To this end, we develop a custom architecture to support the proposed algorithm. Specifically, we design three dedicated pruning engines to effectively support channel pruning, edge pruning, and node pruning respectively. Furthermore, we propose a data-free replication pruning scheme to reduce on-chip memory copy operations and data movement, thereby promoting efficient execution across the engines. These elaborate designs and specialized optimizations effectively translate the theoretical savings achieved through pruning into practical speedup and energy reduction during execution.

To summarize, we make the following contributions:

- We analyze the complex computing paradigm of GNNs and propose an efficient message-passing algorithm that can prune task-independent graph data from multiple granularity. It can significantly reduce the overall complexity of GNN with negligible loss of accuracy.
- We design a novel architecture to effectively support pruning edges/nodes/channels on-the-fly. The elaborate top-k selector and data-free replication pruning scheme effectively translate the theoretical savings into practical speedup and energy reduction.
- Compared to the state-of-the-art GNN accelerator AWB-GCN, MePa achieves on average $1.95\times$ and $2.6\times$ energy-efficiency.

II. Background and Analysis

A. Computing Paradigm of Graph Neural Networks

Recently, there has been growing interest in extending neural network methods to non-Euclidean data [1], [2]. Graph neural networks have been demonstrated to be an important method for learning graph-structured data representations, which have achieved good performance in a wide range of graph-based tasks. Motivated by the success of GNNs, various GNN variants [6], [21], [22], [23], [24] have been proposed, including GCN [6], GAT [22], GraphSAGE [23], etc. Although these variants differ in architecture and target applications, they can be generalized to message-passing algorithms that update node representations by gathering feature messages from neighboring nodes [1]. Table I illustrates the following commonly used notations.

Algorithm 1 illustrates the computation pattern of one message passing layer, and the whole computation process can be summarized into two key operations [10]:

- *Aggregation* operation. For each node v, the feature messages of its neighboring nodes (including self-loop) are aggregated into a single feature vector a_v^l with an **Aggregate** function. Common **Aggregate** functions contain **Mean**, **Average**, **Sum** and **Max**.

$$a_v^l = \mathbf{Aggregate}\left(h_u^l \,|u \in \{N(v) \cup \{v\}\}\right). \quad (1)$$

- *Update* operation. The aggregated feature vector a_v^l is transformed by using a linear transformation (e.g. MLP)

TABLE I
NOTATIONS AND DESCRIPTIONS

Notation	Description
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, H)$	input graph
$\mathcal{V}$	node set of $\mathcal{G}$
$\mathcal{E}$	edge set of $\mathcal{G}$
H	initialized feature matrix
e_{uv}	edge between node u and v
L	number of GNN layer
H^l	feature matrix of layer l
$h_v \in H$	feature vector of node v
a_v	aggregated feature vector of node v
cs	channel importance scores
f_{uv}	node feature similarity
W	weight parameters
$\mathcal{N}(v)$	neighbor set of node v

Algorithm 1: Processing Model Abstraction of One Message Passing Layer.

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, H^l)$;
1: **for** each $v \in \mathcal{V}$ **do**
2: $a_v^l = \mathbf{Aggregate}(h_u^l | u \in \{N(v) \cup \{v\}\})$;
3: $h_v^{l+1} = \mathbf{Update}(a_v^l)$;
4: **end for**
Output: Updated node feature H^{l+1};

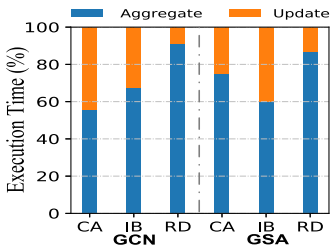


Fig. 2. Execution time breakdown of different methods in two phases for different datasets.

to generate a new node feature h_v^{l+1}.

$$h_v^{l+1} = \mathbf{Update}\left(a_v^l\right). \tag{2}$$

By iteratively executing L of these layers, the final representation of each node captures information about its L-hop neighborhood structure.

Fig. 2 illustrates the execution time breakdown in the two phases for different methods on different datasets.[1] We observe that the execution time of *Aggregation* phase is generally longer than *Update* phase. This is because *Aggregation* phase maintains most of the graph processing behavior, which relies heavily on sparse and irregular graph structures. However, the *Update* phase also occupies a large portion of the execution time on Cora and Citeseer datasets [25] with long feature lengths. Therefore, both *Aggregation* and *Update* take up a significant amount of execution time, and it is necessary to accelerate them.

[1]Refer to Table II for more details of these datasets.

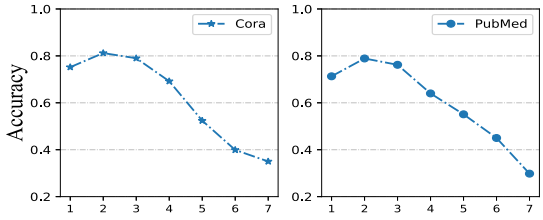


Fig. 3. Testing accuracy of different GCN layers on Cora and PubMed datasets.

B. Opportunities for Efficient Message Passing Model

Due to the high computing complexity of GNNs, it is difficult to deploy GNNs to large-scale or real-time applications. The inefficiency of GNNs comes from several aspects: (1) the real-world graphs are usually very large with complex and irregular neighbor connections, resulting in a prohibitive number of nodes and edges. For example, the Reddit dataset contains a total of 232,965 nodes and 11,606,919 edges, which may lead to inefficiencies in data processing and movement. (2) The dimension of the node features may be very large. For instance, each node in the Citeseer dataset has 3,703 features, which leads to high computing costs for the aggregation stage. Therefore, a straightforward way to improve the efficiency of GNN execution is to reduce the number of nodes, the connections between nodes, and the features of each node. However, naively reducing these parameters can compromise the model performance and thus its achievable task accuracy. This prompts us to answer the question: *Can we prune task-agnostic graph data to speed up GNN execution without losing accuracy?*

1) Redundant Message Passing Resulting in Over-Smoothing: Recent work [26] indicates that not all data are equally important for a specific task. Some redundant and unimportant data can be skipped without compromising or even improving the model accuracy. Graph representation learning is no exception, and only a portion of the graph data, such as some nodes, edges, and feature channels, may be relevant to the task. Nevertheless, existing GNNs [6], [22], [24] follow the specific message-passing pattern, which performs the same message-passing operation for all data. This would introduce redundant message-passing and impair the learning representation capability of GNN. In particular, the performance of GNNs decreases dramatically with the increasing number of layers, which is the notorious over-smoothing [27], [28] problem in GNNs.

We conduct example experiments on the Cora and PubMed datasets with the GCN model, as shown in Fig. 3. Obviously, as the number of layers increases, the classification accuracy of GCN gradually reaches peaks, but the accuracy drops sharply if more layers continue to be stacked. The key factor causing this phenomenon is that the naive implementation of the message-passing model leads to the over-propagation of task-irrelevant data, which introduces redundant message-passing. Specifically, task-irrelevant data require only simple computation, and too much message passing is detrimental to its representation learning instead. Meanwhile, these irrelevant data should not be overly involved in message passing since they may affect the holistic information interaction as noise. We analyze this issue

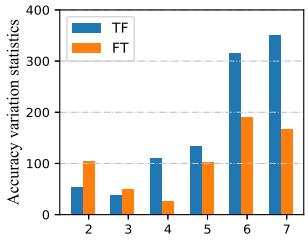


Fig. 4. Accuracy variation statistics. TF denotes vertices shifted from correct to misclassified, and FT denotes vertices shifted from misclassified to correct.

from three aspects: node redundancy, edge redundancy, and feature channel redundancy.

Channel Redundancy: Recently, many works [29], [30] on channel pruning have sprung up to reduce the computational cost of CNNs. The intuition underlying them lies in the fact that images contain numerous task-irrelevant input features and can be channel pruned to allow the network to prioritize the important feature channels and ignore the less important channels. Motivated by this, we argue that a comparable issue arises in GNNs, where extensive task-irrelevant feature channels may exist in node features. By removing these feature channels, it is possible to improve the efficiency of both aggregation and update, and in particular to significantly reduce the computationally intensive matrix-vector multiplication in the updating phase. However, the channel pruning approach has not yet attracted the attention of GNN-related researchers, which raises another opportunity to accelerate GNNs.

Node Redundancy: Different nodes may require different degrees of message passing in GNN. Some nodes will converge in the early layers and generate correct predictions, while the remaining misclassified nodes may require more message-passing processes to obtain the best representation. If more GNN layers are stacked, redundant computations are introduced into the nodes that have already converged, resulting in reduced model accuracy. Fig. 4 illustrates the number of nodes from incorrect to correct classification and the number of nodes from correct to incorrect classification in GNN at each layer. It can be observed that after some layers, the number of nodes switching from correct to incorrect classification increases rapidly. This is because these converged nodes lose their specific information due to redundant propagation, turning from correct classification to incorrect classification.

Edge Redundancy: Furthermore, there is tremendous edge redundancy in the neighbor aggregation phase [31]. We consider that not all neighbors of a node contain task-relevant information in the neighborhood aggregation phase. The feature messages from neighboring nodes can be either gainful information or harmful noise. Therefore, GNN works properly when it receives more useful information than noise, while over-interaction with task-irrelevant neighbors can cause the learned representations to become non-linearly distinguishable. At this time, these edge connections can lead to redundant computations.

2) Opportunities: The above analysis provides significant opportunities to improve the efficiency of GNN execution: (i) Evaluating the importance of feature channels and adaptively selecting the important feature channels for computation. (ii) Distinguishing and pruning task-irrelevant nodes and edges to eliminate redundant message passing. The resulting gains come from several aspects: (i) More computational budget can be dynamically allocated to important data and smaller budget to unimportant data, thus reducing the total inference overhead while maintaining accuracy. (ii) The elimination of computing redundancy can alleviate the over-smoothing issue of GNN, and even improve the learning capability of the model.

III. Related Work

Efficient GNNs: To improve the execution efficiency of GNNs, many researchers [32] have been devoted to developing efficient GNNs. One feasible approach is simplifying the GNN model, which reduces its complexity by modifying and simplifying the model architecture [33], [34], [35]. For instance, SGC [33] reduced model complexity by eliminating the nonlinearity of internal layers. [35] attempted to introduce quantization into GNNs, which employ low-precision integer operations to train GCNs without sacrificing classification accuracy. However, the model framework of GNNs is inherently simple with fewer model parameters. Therefore, these methods have limited gain space and fail to address the fundamental issue of high computational complexity in GNNs.

Another feasible solution is pruning the graph structure that speeds up GNN execution by pruning the number of nodes or edges. (i) Some studies [23], [36], [37] explored various sampling-based strategies to prune task-irrelevant nodes. Their main idea is to sample important nodes in the graph to acquire a subgraph for subsequent training and inference. (ii) Some other studies [15], [16], [17], [18], [19], [20] explored graph sparsification methods to prune graph edges. For instance, Dropedge [15] found that the input graph may contain numerous task-irrelevant edges, and alleviated the over-smoothing problem by randomly removing edges with each training epoch. Building on this, DARe [16] designed a ReRAM-based 3D multicore architecture leveraging two regularization techniques (DropEdge and Dropout) to accelerate GNN training. Note that these methods only work for GNN training and are not aimed at accelerating inference. Furthermore, [17] and [18] further designed a learnable sparse regularizer to dynamically prune task-independent edges. Unlike DropEdge, which only randomly removes edges during training, these methods can apply pruning techniques during testing as well, exhibiting better generalization performance. However, these approaches predominantly focus on pruning the graph structure (nodes or edges) from a singular perspective, lacking a unified framework to comprehensively identify and prune task-irrelevant graph data.

In contrast, our proposed MePa considers an unexplored and orthogonal perspective to prune task-irrelevant graph data at multiple granularity levels: node-wise, edge-wise, and channel-wise, significantly reducing GNN inference cost while providing comparable or even better accuracy.

GCN Inference Acceleration: Many recent studies [13] have been devoted to devising dedicated accelerators for GNNs that achieve high performance through customized hardware. The

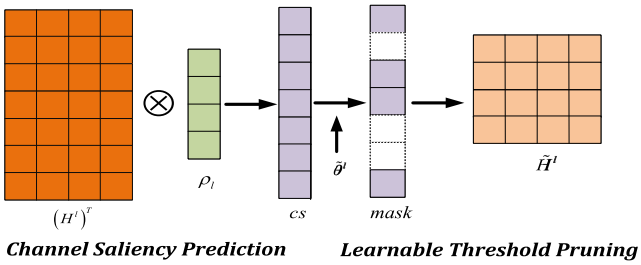


Fig. 5. Illustration of Dynamic Channel Pruning. The example graph contains 4 nodes, each node with 7 feature dimensions. In the channel saliency prediction stage, we utilize the node feature matrix $H^l \in \mathbb{R}^{4\times7}$ to predict the importance score $cs \in \mathbb{R}^{1\times7}$ of each feature channel. Subsequently, learnable threshold pruning is used to retain the 4 channels with the highest scores to generate the pruned node features $\tilde{H}^l \in \mathbb{R}^{4\times4}$.

pioneering work HyGCN [11] decoupled the aggregation and update phases, and designed two specialized computing engines to explore intra-vertex/inter-vertex parallelism. On this basis, ENGN [12] proposed the Ring-Edge-Reduce (RER) dataflow, which can handle graphs of any dimension and improves the scalability of the accelerator for large graphs. AWB-GCN [14] discovered the workload imbalance in the aggregation stage and proposed a uniform sparse matrix multiplication engine to improve hardware utilization. GCNAX [38] proposed a flexible GCN architecture that supports flexible data flows to improve resource utilization. ReGNN [39] proposed a dynamic redundancy elimination architecture by reusing the common neighbors in the graph to improve the execution efficiency of GNN. Although existing accelerators have achieved considerable performance and energy-efficiency improvements for GNNs, these hardware-specific optimization techniques fail to address the fundamental problem of high computational complexity caused by redundant data in GNNs. In contrast, our MePa designs a unified pruning architecture through an algorithm-accelerator co-design approach to prune task-independent graph data at multiple granularity levels to reduce redundant message-passing and improve overall execution efficiency.

IV. Algorithm Design

MePa is an algorithm and hardware co-design accelerator. In this section, we first present the proposed efficient message-passing algorithm that prunes task-irrelevant graph data from multiple granularity levels to reduce redundant message-passing. The details are described as follows.

A. Dynamic Channel Pruning

Based on the analysis in Section II-B, there are plenty of less significant channels in graph data. Motivated by this, we propose a dynamic channel pruning strategy that adaptively selects significant feature channels to reduce the model complexity. Intuitively, it allows GNNs to prioritize certain significant feature channels and skip irrelevant channels to accelerate expensive GNN operations. As shown in Fig. 5, it contains two main processes: channel saliency prediction and learnable threshold pruning.

1) Channel Saliency Prediction: There is an inescapable problem that needs to be addressed before pruning: *How to find the task-relevant data dynamically at runtime?* Towards these goals, we design a channel saliency prediction algorithm to identify the important feature channels. Motivated by [40], we directly adopt the fully connected layer to predict the importance of the feature channels. Specifically, given the feature matrix $H^l \in \mathbb{R}^{n\times C}$ with n node and each node contains C features, the importance of each feature channel can be calculated as:

$$cs = \left(H^l\right)^T \rho_l / \left\|\rho_l\right\|, \tag{3}$$

where $\rho_l \in \mathbb{R}^{n\times1}$ is the learnable weight parameter. $cs \in \mathbb{R}^{1\times C}$ means the importance of the all feature channel, and $cs_i \in cs$ denotes the importance of i-th feature channel. If cs_i is larger, the corresponding feature channel is more important. In this way, we can adaptively learn the importance of feature channels in a task-driven manner, while avoiding the enormous computational costs.

2) Learnable Threshold Pruning: Once the channel importance is determined, the pruning operation can be performed. We first discuss and analyze two common solutions and their drawbacks: top-k pruning and pre-defined threshold pruning. Motivated by this, we propose a novel learnable threshold pruning method, which adaptively removes the task-irrelevant channels with learnable thresholds.

Top-k Pruning: Given the channel importance cs, a straightforward way is to sort the importance scores and output the top-k elements directly. However, the naive implementation has several obvious drawbacks: (i) The top-k pruning is not hardware-friendly, and its pruning occurs after the importance scores of all data have been computed, which greatly limits the execution efficiency of the model. (ii) It requires sorting the importance scores of all channels. Therefore, it is necessary to choose a suitable sorting algorithm and design a dedicated sorting engine to support it. For example, SpAtten [41] elaborately designs a specialized top-k engine to support the selection of the largest k elements.

Pre-defined Threshold Pruning: Different from top-k pruning, pre-defined threshold pruning [42] directly pre-defines a threshold θ and retains channels with cs_i larger than the threshold. Specifically, we generate a binary mask matrix idx. If the cs_i is below the threshold, $\mathrm{M}_i^l = 0$. Conversely, if the cs_i is above the threshold, $\mathrm{M}_i^l = 1$. Formally, M^l is defined as follows:

$$\mathrm{M}_i^l = \begin{cases} 1, & cs_i \geq \theta, \\ 0, & cs_i < \theta. \end{cases} \tag{4}$$

Compared to top-k pruning, such a method requires only a simple comparison operation to implement pruning operations in real-time, thus avoiding the expensive sorting operation and making it computationally very efficient. However, the limitation of this approach is that the overhead of finding the optimal threshold is very expensive. In particular, thresholds vary from layer to layer, and it is necessary to find the optimal threshold for each layer, which makes the search space for the optimal threshold explode. Furthermore, the model accuracy is sensitive to the selection of thresholds, and once the optimal thresholds are not found, the model accuracy decreases dramatically.

Our Solution: To address the above issues, we propose a learnable threshold pruning scheme, which allows the network to employ gradient-based back-propagation methods to adaptively learn the threshold. Specifically, we employ the $\mathrm{sigmoid}(\cdot)$ function to transform the non-differentiable mask $\widetilde{\mathrm{M}}_i^l$ into a differentiable soft mask:

$$\widetilde{\mathrm{M}}_i^l = \mathrm{sigmoid}\left(\frac{cs_i - \tilde{\theta}^l}{T}\right), \tag{5}$$

where $\tilde{\theta}^l$ is the learnable threshold that is initially randomly assigned by the network. With the progress of training, $\tilde{\theta}^l$ will be adaptively adjusted by itself with the change of loss value. T is the pruning influence factor that can amplify the relationship between similarity score and learnable threshold. With the effect of T, the channels with cs_i greater than $\tilde{\theta}^l$ is approximately 1 in $\widetilde{\mathrm{M}}^l$. Conversely, the channels with cs_i less than $\tilde{\theta}^l$ is approximately 0 in $\widetilde{\mathrm{M}}^l$.

Finally, we multiply the soft mask $\widetilde{\mathrm{M}}^l$ by the feature matrix H^l to obtain the pruned feature matrix $\tilde{H}^l$:

$$\tilde{H}^l = \widetilde{\mathrm{M}}^l \cdot H^l. \tag{6}$$

This approach ensures that the channels in the updated feature matrix $\tilde{H}^l$ with cs_i much less than $\tilde{\theta}^l$ are close to zero, and thus these channels have little impact on subsequent layers. Therefore, the novel learnable pruning scheme is almost identical in behavior to threshold pruning, but its differentiable form allows the model to learn the weight parameters and the threshold simultaneously.

Light Fine-tuning Strategy: Jointly training GNN models and learnable thresholds in an end-to-end manner may affect model performance. This is because improper thresholds may be learned during the early training phase, thereby impairing the entire learning process. To address this issue, we propose a light fine-tuning training strategy to stabilize the learning of thresholds: (i) Pretrain the GNN model without applying threshold pruning. It is easier to learn meaningful thresholds based on a well-performing pretrained model. (ii) Combine the pretrained model with the proposed learnable threshold scheme for joint training to fine-tune the model parameters and learn the pruning thresholds for each layer. (iii) Fix the learned thresholds, binarize the soft mask $\widetilde{\mathrm{M}}^{(l)}$ with (4), and perform fine-tuning to the model parameters.

During the inference phase, we directly utilize the learned thresholds to prune the channels like (4). With this learnable threshold pruning, we combine the advantages of top-k pruning and predefined threshold pruning to efficiently support runtime pruning: (i) Similar to predefined threshold pruning, learnable threshold pruning avoids costly sorting operations and only requires a simple comparison operation for efficient pruning operations. (ii) Learnable thresholds adaptively find the optimal thresholds via a gradient-based backpropagation algorithm without relying on limited empirical evidence.

Differentiable Regularization: Using task loss as the sole constraint to train the model, the network may fail to learn the threshold. Since all channels are present, the optimizer generally obtains a better loss value. It implies that the network tends to lower the threshold and retain more channels. To this end, we add a regularization term to force the network to remove task-independent channels. This is achieved by applying L_1 regularization to the mask matrix $\widetilde{\mathrm{M}}^{(l)}$:

$$\mathcal{L} = \mathcal{L}_{task} + \alpha \mathcal{L}_{reg}^{(l)} \quad \text{where} \quad \mathcal{L}_{\mathrm{reg}} = \frac{1}{L}\sum_{l=1}^{L} \|\widetilde{\mathrm{M}}^l\|_1, \tag{7}$$

where $\mathcal{L}_{task}$ is the task-specific loss and $\widetilde{\mathrm{M}}^l$ is the mask matrix of the l-th layer, which records the number of points reserved by the current network. If the learnable threshold $\tilde{\theta}^l$ is too small, the more nodes are retained in $\widetilde{\mathrm{M}}^l$, and the value of $\mathcal{L}_{reg}$ is larger. By minimizing the loss function, the threshold can be driven to a larger value, and more task-independent channels can be removed near the threshold boundary. α is the loss value influence factor. Larger values of α result in higher pruning ratios.

B. Dynamic Edge/Node Pruning

Following the analysis in Section II-B, there are numerous node redundancy and edge redundancy in GNN. These redundant nodes and edges involved in too much message passing would cause the node representations to become increasingly similar and finally converge to similar indistinguishable vectors. To this end, we propose dynamic edge/node pruning to identify and eliminate redundant nodes and edges.

1) Feature Smoothness: We define a unified quantitative metric: *feature smoothness*, which estimates which nodes or edges need to be pruned. Specifically, given the feature vectors of node v and node u, the feature smoothness f_{uv} is calculated as follows:

$$f_{uv} = \|h_u - h_v\|_1 / C. \tag{8}$$

where $\|\cdot\|_1$ is Manhattan distance with low computational cost. f_{uv} is a metric that reflects the similarity among nodes. A smaller f_{uv} indicates that the node features h_v and h_u are more likely similar. If node v and node u are connected, the overly similar neighbor u may not offer useful information to node v. We can remove this edge e_{uv} to eliminate edge redundancy. For nodes, if the feature smoothness of input node feature h_v and aggregated feature a_v is particularly small, the node has converged in advance. We can eliminate the node redundancy by skipping the subsequent aggregation and update operations.

2) Edge Pruning: Based on the feature smoothness, the execution flow of edge pruning is as follows. Given the node u and its neighbors set $v \in \mathcal{N}(u)$, we first calculate the feature similarity of vertex u and all its neighbors $\mathcal{N}(u)$ by (8), and subsequently utilize the proposed learnable threshold pruning to retain the valuable edges. Once edges are removed, they are not present in the following message-passing layer.

3) Node Pruning: Similar to edge pruning, we first utilize the input node feature h_v^l and the aggregation feature A_v^l to compute the feature smoothness, and then prune the nodes using the proposed learnable threshold pruning. If the feature smoothness is less than the computed threshold, the node is considered converged and it can be pruned to eliminate vertex redundancy.

Note that in subsequent message passing, the pruned nodes stop aggregating and updating themselves, but can still pass on their features to other nodes.

C. Efficient Message Passing Algorithm With Dynamic Pruning

Based on the above discussion, we present an efficient message-passing algorithm that prunes task-irrelevant graph data from multiple granularities to reduce redundant message passing, i.e., channel/edge/node-wise. As shown in Algorithm 2, for each node $v \in \mathcal{V}$, we first perform dynamic channel pruning on node v and its neighbors $u \in \mathcal{N}(v)$ to generate intermediate node features $(\bar{h}_u^l, \bar{h}_v^l)$. Subsequently, edge pruning is executed on each edge e_{uv} of v with the intermediate node feature. If the edge is reserved, the aggregation operation is performed. After all neighbors of v are processed, the node pruning operation is executed. Finally, for the retained nodes, the update operation is executed to obtain the new representations. To avoid duplicate computations, we store and reuse the intermediate node results generated by channel pruning. Note that the pruned graph data are not considered in subsequent message-passing layers.

Complexity Analysis: The computational complexity of the general GCN model inference can be expressed as $O(L|\mathcal{E}|C + LnC^2)$, where L is the number of GCN layers, n is the number of nodes, $|\mathcal{E}|$ is the number of edges, and C is the feature dimensions of each node.

Compared with the general GCN model, our approach would introduce extra computational overhead: channel pruning $O(nC)$, node pruning $O(nC')$, and edge pruning $O(|\mathcal{E}|C')$. However, channel pruning can squeeze the cost by reducing the number of channels ($C' < C$), while node/edge pruning further squeezes the cost via a smaller number of nodes/edges ($|\mathcal{E}'| < \mathcal{E}|$ and $n' < n$) in the graph. The computational complexity of a GCN layer with dynamic pruning can be expressed as $O(nC + |\mathcal{E}|C' + |\mathcal{E}'|C' + nC' + n'C'^2)$. More importantly, the pruned graph data are not required to participate in the subsequent message-passing, rendering the computational overhead introduced by pruning operations in the subsequent layers negligible. In summary, although dynamic pruning introduces additional computation, it is trivial compared to the gain.

Algorithm 2: The proposed Efficient Message Passing Algorithm.

for l in L **do**
  for each $v \in \mathcal{V}$ **do** ◁ *Channel Prune*
    for each $u \in \mathcal{N}(v)$ **do**
      $cs \leftarrow \mathbf{ImportanceCompute}(h_u)$;
      for $i \in C$ **do**
        if $cs_i \leq \tilde{\theta}_1$ **then**
Prune channel i
        end if
      end for
      Generating intermediate node features $\bar{h}_u$
    end for
    ◁ *Edge Prune*
    for each $u \in \mathcal{N}(v)$ **do**
      $f_{uv} \leftarrow \mathbf{SmoothnessCompute}(\bar{h}_u^l, \bar{h}_v^l)$;
      if $f_{uv} \leq \tilde{\theta}_2$ **then**
        Prune e_{uv}
      end if
    end for
    ◁ *Aggregation*
    for each reserved $u \in \mathcal{N}(v)$ **do**
      $a_v^l = \mathbf{Aggregate}(\bar{h}_v^l, \bar{h}_u^l)$;
    end for
    ◁ *Node Prune*
    $f_{va} \leftarrow \mathbf{SmoothnessCompute}(\bar{h}_v^l, a_v^l)$;
    if $f_{va} \leq \tilde{\theta}_3$ **then**
      Prune v
    else
      Reserve v
    end if
    ◁ *Update*
    if v is reserved **then**
      $h_v^{l+1} = \mathbf{Update}(a_v^l)$;
    end if
  end for
end for

V. Architecture Design

Current GNN accelerators [11], [14], [39] cannot support the proposed dynamic pruning mechanism well. For this reason, we devise a novel architecture, called MePa, which efficiently supports the proposed dynamic pruning mechanism and translates it into performance gains.

A. Architecture Overview

Fig. 6 illustrates the overall architecture of the designed MePa. It is a pipelined architecture accelerator that integrates five core functional engines: *ChPruner*, *EdPruner*, *NoPruner*, *Aggregator*, and *Updater*. These elaborate computation engines are used to support different operations and optimized separately to improve intra-block parallelism. Meanwhile, this fine-grained pipeline architecture can hide the communication latency and further improve the inter-block parallelism.

ChPruner: The custom *ChPruner* is designed to support dynamically pruning channels during inference, which uses a specialized channel pruning processing element (CP-PE) group to dynamically compute channel importance and prune task-independent channels according to importance.

EdPruner/NoPruner: The *EdPruner* and *NoPruner* are used to perform dynamic pruning for edges and nodes, respectively. Despite their different functions, they share the same architecture because their basic calculations are consistent. Similar to *ChPruner*, *EdPruner* and *NoPruner* contain a group of pruning processing elements (FP-PE) for dynamically computing feature smoothness among the nodes and preserving important neighbor/node indexes.

Aggregator: The *Aggregator* is designed to execute aggregation operation tasks. It consists of an aggregation scheduler

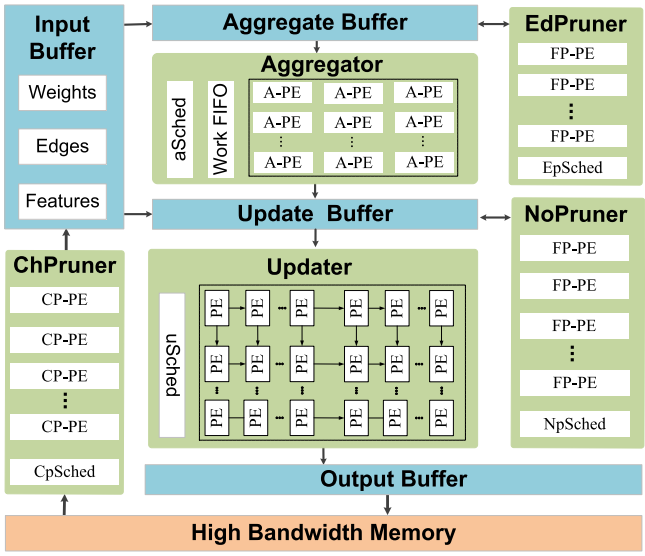


Fig. 6. Architecture Overview of MePa.

(aSched) and an aggregate processing element (A-PE) array. The aSched is responsible for assigning the workload of the aggregation phase, and the A-PE iteratively accesses the relevant features cached in the Aggregate Buffer to perform the information aggregation of the nodes.

Updater: The *Updater* performs feature transformations to complete the update operation. Considering that the update operation contains computation-intensive matrix-vector multiplication (MVM), we adopt the classic systolic array [43] to improve data reusability and processing parallelism.

Buffer Management: To reduce the data access latency, on-chip buffers are employed to cache various data and intermediate results for improving data reuse. The input buffers are designed to store pruned node features, edges, and weights. The aggregation buffer is designed to cache the node features waiting to be aggregated and the required edge information. The update buffer stores the aggregated intermediate results in preparation for the subsequent update operation. The output buffer is used to coalesce the write accesses of the final features. We utilize the double buffering technique for almost all buffers to overlap the data transfer time with computation.

B. Design of ChPruner

Previous GNN accelerators [11], [12] do not consider channel pruning operations, which requires a novel redesign of the architecture. To achieve this, we devise the novel *ChPruner* to support dynamic channel pruning, which is composed of three core components: a task scheduler (*CpSched*) and a group of channel pruning processing elements (CP-PEs). Fig. 7(a) illustrates the workflow of *ChPruner*.

❶ *CpSched* is responsible for assigning the workload of channel pruning to the CP-PEs. It fetches the nodes waiting to be pruned and the corresponding weights ρ_l from the High Bandwidth Memory (HBM).

❷ CP-PEs calculate the importance scores of the feature channels and retain the important channels based on their importance.

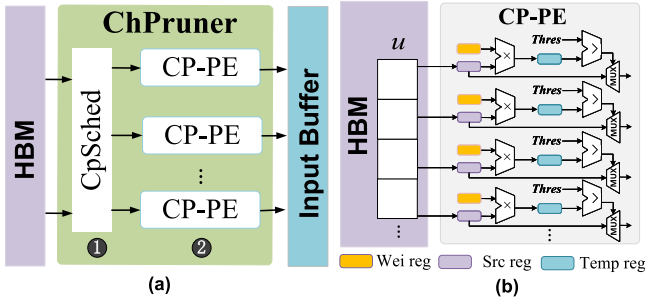


Fig. 7. (a) Workflow of *ChPruner*. (b) Architecture of CP-PE.

As shown in Fig. 7(b), each CP-PE is responsible for pruning the feature channels of a node, which consists of multiple ALUs, registers, and MUX. Given a node u to be processed, each *Src reg* in CP-PE reads the one-dimensional feature of node u, and *Wei reg* reads the weight parameter ρ_l from the HBM. The channel importance score is obtained by executing multiplication operations in parallel with multiple ALUs and caching it to *Term reg*. Subsequently, the channel importance and the learned threshold $\tilde{\theta}_1$ are sent to ALU for a comparison operation to produce a binary bit control signal (0: smaller than $\tilde{\theta}_1$, 1: larger than $\tilde{\theta}_1$). Finally, MUX receives the control signal to determine whether the feature channel is retained. If the control signal is 1, the feature channel contains valuable information and retains it in the input buffer.

In this manner, we load the node features from the HBM only once to complete the channel pruning. Furthermore, it is no longer necessary to design additional sorters to support sorting operations with higher complexity, yet only perform simple comparison operations to achieve efficient pruning.

C. Design of EdPruner

The *EdPruner* aims to support dynamic edge pruning during propagation. A straightforward implementation involves iteratively accessing the cached edge and node features in the aggregation buffer, and writing the pruned results back to the aggregation buffer. Subsequently, the aggregator accesses the stored pruned results to perform the aggregation operation. However, this approach introduces numerous on-chip memory copy operations and data movement. Furthermore, the on-chip memory space is severely limited and may not accommodate these additional overheads, thereby significantly increasing the average memory access latency. Therefore, it is necessary to reduce data movement to uphold efficient execution between engines.

Data-free Replication Pruning Scheme: To solve the mentioned problem, we devise the data-free replication pruning scheme. Fig. 8 depicts the cooperation flow among *EdPruner* and *Aggregator*, which comprises three phases: (i) **Edge Prune**. *EdPruner* generates the neighbor node addresses that require retention through dynamic edge pruning. (ii) **Push work**. The generated addresses are pushed into the *Work FIFO* of the *Aggregator*. (iii) **Aggregate**. Once the *Work FIFO* receives the address, the dispatcher of the *Aggregator* is started immediately. It retrieves the address from the *Work FIFO* and acquires

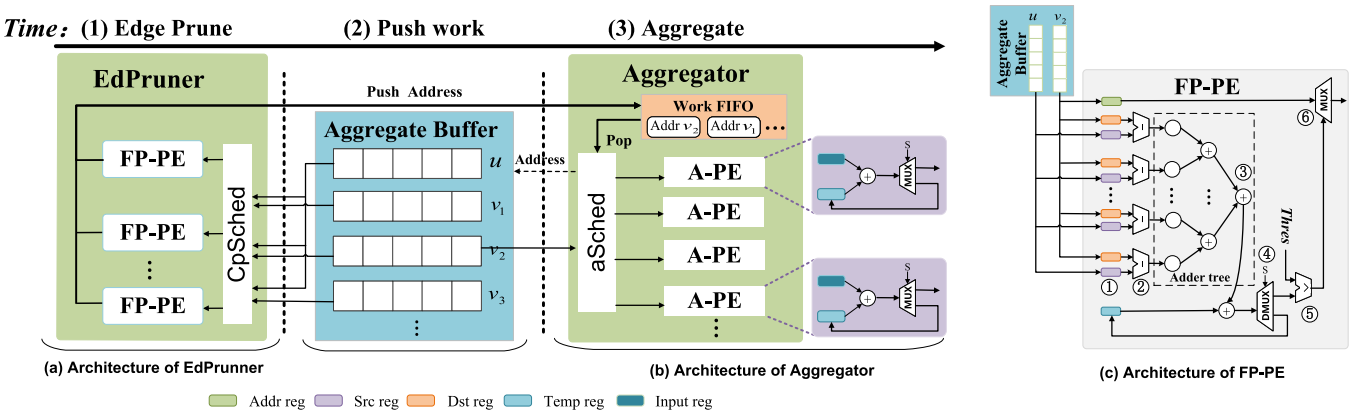


Fig. 8. Cooperation of *EdPruner* and *Aggregator*. (a) Architecture of *EdPruner*. (b) Architecture of *Aggregator*. (c) Architecture of FP-PE.

the neighbors' features for subsequent aggregation based on the address. This approach transforms EdPrune into an address generator, mitigating data replication and movement overheads and enhancing data reuse.

Architecture of EdPruner **:** Fig. 8(a) presents the architecture of *EdPruner*, which includes a task scheduler (*Epsched*) responsible for assigning tasks, a group of pruning processing elements (FP-PE). Similar to the workflow of *ChPruner*, *Epsched* assigns the work of edge pruning to CP-PES, and each CP-PE returns the feature Smoothness between two nodes and performs pruning operations. Fig. 8(d) details the micro-architecture of a single CP-PE, which contains several registers, ALUs, an adder tree, a DMUX, and a MUX. Take an edge e_{uv_2} as the example, the execution process of the single CP-PE is illustrated as follows:

① The *Addr reg* caches the address of node v_2. Each *Dst reg* and *Src reg* fetch the single-dimension features of node u and v_2 from the aggregation buffer.

② Leverages multiple ALUs to perform subtraction operations in parallel for intermediate results.

③ The adder tree collects the intermediate results and conducts the accumulation operations in parallel to obtain the intermediate smoothness.

④ DMUX is applied to judge whether all dimensions have been handled. If DMUX outputs 0, intermediate smoothness is stored in Temp reg and the ①, ②, and ③ steps are repeated until all dimensions are processed.

⑤ Send the channel importance and the learned threshold $\tilde{\theta}_2$ to ALU for a comparison operation to produce a binary bit control signal (0: larger than $\tilde{\theta}_2$, 1: smaller than $\tilde{\theta}_2$).

⑥ MUX receives the control signal to determine whether the edge is retained. If the control signal is 1, push the address stored in *Addr reg* to *Work FIFO*.

In this manner, we can process multiple feature dimensions in parallel that fully exploit intra-node parallelism. Meanwhile, FP-PE supports processing nodes with arbitrary feature dimensions. When the dimensions that can be processed by the adder tree in FP-PE cannot cover the feature dimensions of the nodes, we utilize *Temp reg* to cache the uncomputed intermediate results. The remaining feature dimensions with processing continue to be inserted into FP-PE until all feature dimensions are processed.

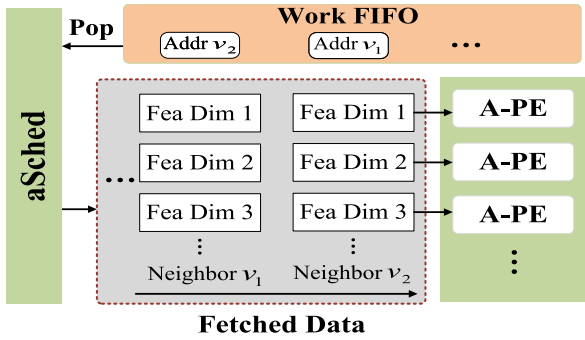


Fig. 9. Workflow of *Aggregator*.

D. Design of Aggregator

The aggregation process involves gathering the neighbor information according to edge connections. Typically, two processing modes are employed: task-centralized and task-decentralized. In the task-centralized mode, each node's workload is assigned to a single A-PE. However, this mode is hindered by graph irregularity and results in workload imbalances. Due to the different number of neighbors per node, nodes with fewer neighbors need to wait for nodes with many neighbors, which causes high processing latency of nodes. To address this issue, we employ the task-decentralized mode, similar to the approach used in HyGCN, to achieve workload balancing.

Fig. 9 depicts the workflow of the *aggregator*. During each clock cycle, a neighbor address to be aggregated is popped from the *Work FIFO*, and the neighbor features are fetched from the aggregation buffer using *aSched* based on the address. Each A-PE is responsible for aggregating single-dimensional properties. In particular, the *Input reg* in each A-PE sequentially reads the one-dimensional property of the fetched neighbor features and performs feature accumulation. The accumulated intermediate results are stored in the *Temp reg*. The *Temp reg* is emptied once all the neighbors of the vertex have been processed. In cases where the dimension of a node cannot utilize all the available A-PEs, the idle A-PEs are allocated to other nodes. In this way, the A-PE group can handle nodes with arbitrary dimensional attributes. Moreover, all A-PEs remain occupied all the time and no idle computational resources are caused.

TABLE II
DATASET STATISTICS

Dataset	Label	$\#\mathcal{V}$	$\#\mathcal{E}$	Feature Dim.
Cora (CA)	7	2708	10 556	1433
Pubmed (PB)	3	19 717	88 648	500
Citeseer (CS)	6	3327	9104	3703
Ogbn-Arxiv (OA)	40	169 343	1 166 243	128
Ogbn-Proteins (OP)	2	132 534	39 561 252	8
Reddit (RD)	40	232 965	114 615 892	602

TABLE III
COMPARISON AMONG MEPA AND THE BASELINE MODELS IN TERMS OF ACCURACY

Methods	Cora	Citeseer	Pubmed	Ogbn-ArXiv	Ogbn-Proteins	Reddit
GCN-Vanilla	81.5	70.3	79.0	71.7	72.5	93.7
Dropedge	81.6	72.2	78.5	70.6	72.7	96.1
NeuralSparse	82.1	71.5	78.8	-	-	96.3
PTDNet	82.8	**72.7**	**79.8**	-	-	**96.4**
UGS	81.9	71.5	79.2	70.2	73.5	96.0
MePa (Our)	**83.0**	72.2	79.5	**71.9**	**73.9**	96.2
GraphSAGE-Vanilla	79.2	67.6	76.7	71.5	77.7	93.8
Dropedge	78.7	67.0	74.8	71.2	76.9	96.3
NeuralSparse	79.3	67.4	75.1	-	-	**96.7**
PTDNet	80.3	67.9	77.1	-	-	96.2
UGS	82.3	69.0	77.8	70.5	77.9	94.3
MePa (Our)	**82.5**	**69.5**	**78.5**	**72.2**	**78.2**	96.6
GAT-Vanilla	83.0	72.1	79.0	73.2	80.1	**96.7**
Dropedge	83.2	70.9	77.9	73.3	78.9	96.5
NeuralSparse	83.4	72.4	78.0	-	-	95.9
PTDNet	**83.7**	**73.4**	79.3	-	-	95.6
UGS	83.1	71.9	78.5	72.5	80.6	96.1
MePa (Our)	83.2	73.4	**79.2**	**73.5**	**81.9**	96.0

The bold values indicates that the performance achieved by the method is optimal.

VI. EXPERIMENTAL RESULTS

A. Algorithm Evaluation

1) Datasets: We conduct experiments on six popular graph datasets to validate the proposed MePa, including three common citation datasets (Cora, Pumbed, and CiteSeer), two large-scale graph datasets from Open Graph Benchmark (OGB), and Reddit dataset. Table II summarizes the detailed statistics of these datasets.

2) GNN Models and Baselines: We conduct experiments with the 3 most common GNN models: GCN, GraphSAGE, and GAT. We adopt Pytorch Geometric (PyG) [44] to implement the models with the default configuration. Moreover, we compare our method with four representative graph sparsifiers: (i) Dropedge [15] that randomly drops edges, (ii) NeuralSparse [17] and PTDNet [18] both use learnable sparse regularizer to adaptively prune edges, (iii) UGS [20] that utilizes the lottery ticket hypothesis to simultaneously sparsify the input graph and the GNN model.

3) Implementation Details: To ensure a fair comparison, we use the same network architecture as described in [6]. The network depth is set to 3, and the dimension of the feature vector in each layer is set to 512. We apply dropout of $p = 0.5$ for the input in each layer, and utilize LeakyReLU as the activation function. We adopt Adam optimization to train proposed models, the learning rate is 0.1, and the weight decay is 0.0005, and the maximum of training iterations is 200 epochs.

4) Model Performance: We compare our proposed MePa with the original models (Vanilla) and the baseline models [15], [17], [20] across different models and datasets. Table III reports the accuracy results for each model. We can easily observe the following aspects: (i) The proposed MePa can match or outperform the accuracy of all baseline models in different scenarios. Overall, the accuracy decreased by no more than 0.7% after dynamic pruning in all cases. Remarkably, MePa improves accuracy on most datasets, especially Mepa (GAT) improves its accuracy by 1.8% on the Ogbn-Proteins dataset. It effectively demonstrates that the proposed MePa can achieve comparable or even superior accuracy while reducing the model complexity of GNNs. (ii) Unlike DropEdge, which only removes edges randomly in the training phase, PTDNet, NeuralSparse, and UGS can dynamically remove task-irrelevant graph edges or model parameters using parameterization methods in both the training and inference phases to achieve better performance. Compared to these approaches, MePa is a unified framework that prunes task-irrelevant graph data at multiple granularity levels: node-wise, edge-wise, and channel-wise. This fine-grained pruning approach drastically reduces redundant message passing, improves the quality of message passing, and achieves better generalization performance.

5) Pruning Ratio Exploration: Since the channel/edge/node pruning ratio directly affects the performance of the proposed method, we explore the variation of model accuracy under different pruning ratios. Specifically, we search for pruning influence factor T in a search space of $\{1, 2, 5, 10, 20\}e-4$ and vary α from 0.001 to 0.4 to control for different pruning ratios. Fig. 10 shows the accuracy of the GCN model for each dataset with different channel/node/edge pruning ratios (the x-axis indicates the pruning ratio and the y-axis indicates the accuracy).

Channel Pruning Ratio: We have the following observations from Fig. 10: (i) Initially increasing the channel pruning ratio does not compromise the model accuracy, and even improves the accuracy on most datasets. Remarkably, on the Cora dataset with a large initial feature dimension, accuracy improves by 2.2% with a channel pruning ratio of 43%. This improvement may be attributed to appropriate channel pruning, eliminating redundant message interactions introduced by task-independent feature channels, thereby reducing noise interference and enhancing the model's representation learning ability. (ii) Excessively increasing the pruning ratio leads to a significant loss of accuracy. This is because pruning too many feature channels causes information loss. (iii) Different datasets have different optimal channel pruning ratios. In particular, there is no loss of model accuracy for the Cora and Citeseer datasets with large initial feature dimensions when the average channel pruning ratio reaches 63%. (iv) The proposed MePa consistently outperforms the random pruning method on all datasets and becomes more pronounced as the channel pruning ratio increases. This is because our MePa allows the model to pay more attention to those important feature channels, ensuring high-quality message passing.

Node Pruning Ratio: For node pruning, we have similar observations: (i) A suitable node pruning ratio can improve the accuracy of the original model across all datasets. This is because node pruning can terminate converged nodes early to participate in too much message passing, effectively improving the quality of message passing. (ii) Different datasets have different optimal node pruning ratios. For datasets with a large number of nodes (Ogbn-Arxiv, Obgn-Proteins, and Reddit), pruning more nodes is feasible. Specifically, on the Ogbn-Arxiv dataset, up to 61%

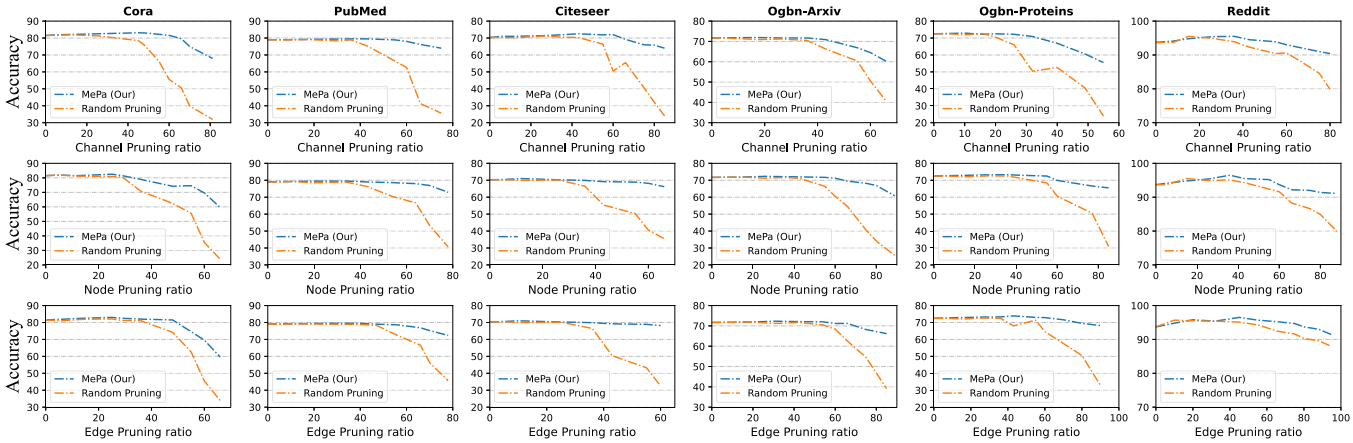


Fig. 10. Accuracy of GCN model for each dataset with different pruning ratios.

TABLE IV
SYSTEM CONFIGURATIONS OF MEPA AND BASELINES

	HyGCN	EnGN	AWB-GCN	GCNAX	MePa
Compute Unit	1GHz @ 32 SIMD 16 cores and 32×128 arrays	1GHz @ 128×16 arrays and 32 PE units in VPU	1GHz @ 4K PEs	1GHz @ 1×16 MAC array	1GHz @ 512 A-PEs, 64 CP-PEs, 32 FP-PEs, and a systolic module with 32×128 arrays
On-chip Memory	22MB+128KB	1600KB	12MB	4MB	16MB
Off-chip Memory	256GB/s HBM 1.0	256GB/s HBM 1.0	256GB/s HBM 1.0	256GB/s HBM 1.0	256GB/s HBM 1.0

of nodes can be pruned without any accuracy loss. (iii) In terms of node pruning, MePa consistently outperforms the random pruning approach on all datasets.

Edge Pruning Ratio: Similar to the above results, different datasets exhibit different optimal edge pruning ratios. The high-degree Reddit dataset can sustain an 80% pruning ratio without sacrificing accuracy. Furthermore, an appropriate edge pruning ratio can notably enhance model accuracy. This can be attributed to the fact that edge pruning removes noisy edges that do not bring gains, effectively eliminating redundant message passing. Finally, in terms of edge pruning, MePa also consistently outperforms random pruning methods on all datasets.

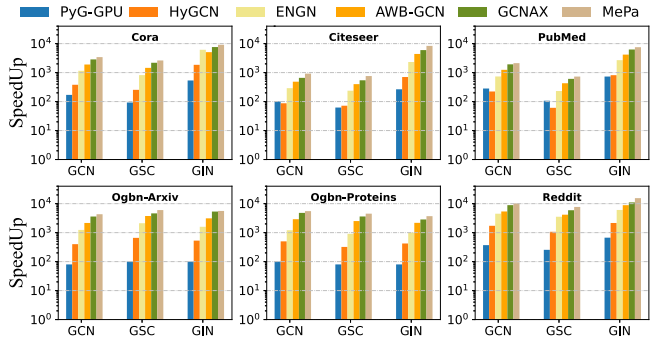


Fig. 11. Speed comparison over PyG-CPU.

B. Architecture Evaluation

1) Hardware Implementation: We implement the modules of MePa with Verilog, and the synthesis of MePa is conducted using Synopsys Design Compiler (DC) with the TSMC 40 nm standard library. Power consumption and area of the logic component are derived from the synthesis results. For the on-chip buffers, their power and area were estimated using CACTI [45]. To model the off-chip HBM behavior, we employ Ramulator [46] to simulate memory operations for estimating HBM clocking, and generating instruction traces.

2) Compared Baselines: To evaluate the performance, we compare MePa with two different hardware platforms: (1) The general-purpose processors, including CPUs (Intel Xeon(R) CPU E5-2680 v3 CPUs) and GPUs (NVIDIA Tesla V100). To leverage the full capabilities of the general-purpose processors, we employ the best-optimized framework Pytorch geometric (PyG) to deploy GNN models. We refer to the CPU platform running PyG as PyG-CPU and the GPU platform running PyG as PyG-GPU. (2) The state-of-the-art GNN accelerators: HyGCN [11], EnGN [12], AWB-GCN [14], GCNAX [38]. To facilitate fair performance comparisons, we enhance the configuration of the baseline accelerators to match the requirements of Mepa. Meanwhile, we extend our Mepa to support the dataflow of the baseline accelerators. The detailed configurations of the hardware platforms mentioned above can be found in Table IV.

3) Overall Results: We comprehensively evaluate MePa from various aspects:

Speedup: Fig. 11 illustrates the speed comparison between MePa and other baselines. We measure the overall execution cycle count as a performance metric. On average, MePa achieves an impressive speedup of 5315× compared to PyG-CPU and 47.1× compared to PyG-GPU. This significant improvement highlights

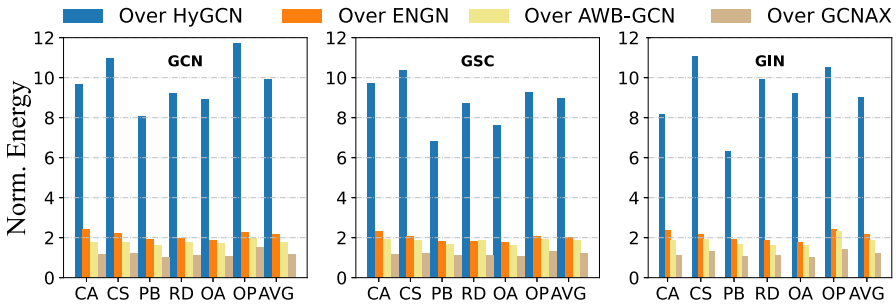


Fig. 12. Energy savings of MePa over the baselines.

the effectiveness of designing a specialized architecture tailored to GNN. MePa leverages the pipelined dataflow to optimize the inter-stage parallelism, resulting in enhanced performance.

For different accelerator baselines, MePa is on average $9.6\times$, $3.3\times$, $1.95\times$, and $1.25\times$ faster than HyGCN, EnGN, AWB-GCN, and GCNAX respectively. This is because the proposed MePa prunes task-irrelevant graph data at multiple granularity levels, dramatically reducing the computational complexity of the GNN. Meanwhile, the pruned graph data does not participate in subsequent message passing, which not only reduces redundant computation, but also reduces off-chip and on-chip memory access overhead. Although *ChPruner*, *EdPruner* and *NoPruner* introduce additional importance or similarity calculations, it is trivial compared to the improvement. Moreover, the proposed data-free replication pruning scheme further reduces the on-chip data movement and decreases the latency time introduced by the extra pruning operations.

DRAM Access: MePa reduces the DRAM accesses by $9.8\times$, $3.4\times$, $1.9\times$, and $1.4\times$ on average compared to HyGCN, EnGN, AWB-GCN and GCNAX respectively. This is because MePa can significantly reduce DRAM read operations brought about by redundant graph data by dynamically pruning channels/edges/nodes. In particular, due to the unique computational paradigm of GNN, some nodes and edges require repeated reads. Moreover, the pruned channels/edges/nodes do not involve subsequent message interactions, thus further saving off-chip memory access overhead.

Energy Consumption: Fig. 12 illustrates the normalized energy efficiency comparison of MePa with other accelerators. MePa demonstrates superior energy performance, consuming only $10.3\times$, $3.4\times$, $2.6\times$, and $1.21\times$ compared to HyGCN, EnGN, AWB-GCN, GCNAX, respectively. The improvement mainly arises from several aspects: (1) Our MePa inherently reduces the computing complexity of the GNN itself through dynamic edge/channel/node pruning, which greatly reduces the computational workload in the *aggregation* and *update* phases. This reduction is particularly prominent in the calculation-intensive MVM (Matrix-Vector Multiplication) operations of the *Update* phase. (2) MePa saves energy by reducing off-chip memory accesses. The pruned channels and edges are not considered in the subsequent message-passing layer, which greatly saves the overhead of off-chip memory reads and writes. (3) The proposed data-free replication pruning scheme improves data reuse and reduces redundant access to on-chip memory.

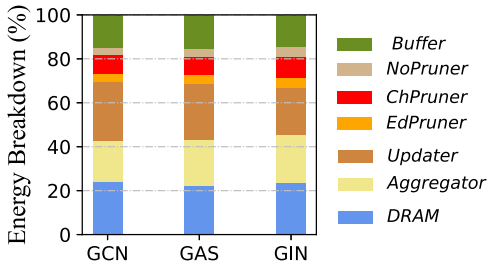


Fig. 13. Energy breakdown of MePa.

TABLE V
AREA BREAKDOWN OF MEPA

Module	Area (%)
Updater	45.84
Aggregator	17.84
TotalBuffer	32.72
ChPruner	2
EdPruner	0.8
NoPruner	0.8

Fig. 13 depicts the energy breakdown of MePa on different models and datasets. Since the differences in data and models, the energy breakdown is different at each stage. However, power-hungry DRAM accesses, *Aggregator* and *Updater* consume the most energy in all cases. The designed *EdPruner*, *ChPruner* and *NoPruner* consume only a very small fraction of the energy, which indicates that the additional computational overhead introduced by the pruner is insignificant compared to the performance improvement.

Area Overhead Breakdown: The total area occupied by MePa is about 12.5 mm^2. Table V demonstrates the area breakdown of MePa's core components. Among them, The *aggregator* and *updater* occupy most of the area of MePa, more than 50% of the total area, since they are the two most computationally intensive modules. Then, the on-chip buffers are responsible for caching data and intermediate results, which consume approximately 33% of the area. In comparison, the *ChPruner*, *NoPruner*, and *EdPruner* components exhibit a mere 3.6% area overhead. These results indicate that the pruner design imposes only a small area overhead. This benefits from the proposed learnable threshold pruning, which efficiently supports pruning without introducing additional sorters. Despite the introduction of some

TABLE VI
EFFECTIVENESS OF THE LEARNABLE THRESHOLD PRUNING

Methods	Cora		Ogbn-ArXiv		Reddit	
	Acc (%)	Speedup (×)	Acc (%)	Speedup (×)	Acc (%)	Speedup (×)
$\text{MePa}_{\text{topk}}$	82.6	7.2	71.7	8.2	**96.7**	9.0
MePa_{pre}	72.6	9.6	55.5	10.8	92.0	12.1
MePa	**83.0**	**9.6**	**71.9**	**10.8**	96.2	**12.1**

The bold values indicates that the performance achieved by the method is optimal.

TABLE VII
COMPARISON AMONG CHANNEL-WISE, NODE-WISE AND EDGE-WISE PRUNING AT DIFFERENT GRANULARITIES IN MEPA

Methods	Cora		Ogbn-ArXiv		Reddit	
	Acc (%)	Speedup (×)	Acc (%)	Speedup (×)	Acc (%)	Speedup (×)
MePa_{CP}	82.7	4.6	71.6	3.5	96.5	4.2
MePa_{EP}	83.2	1.21	72.0	1.48	96.8	1.41
MePa_{NP}	83.1	1.63	71.8	1.91	96.7	2.12
MePa	83.0	9.6	71.9	10.8	96.2	12.4

area overhead, the computation and memory access costs of other computational engines, particularly the expensive aggregators and updaters, are substantially mitigated. The additional overhead is trivial for the obtained performance gain.

C. Ablation Study

Effect of Learnable Threshold Pruning: To demonstrate the effects of the proposed learnable threshold pruning, we explore two variants: MePa_{pre} denotes introducing a predefined pruning threshold to replace the learnable threshold, and $\text{MePa}_{\text{topk}}$ denotes using top-k pruning. Table VI presents the experimental results of GCN on three graph datasets, including model accuracy and speedup (compared to HyGCN). We can observe that $\text{MePa}_{\text{topk}}$ achieves comparable accuracy to MePa, but its execution efficiency is much lower than MePa with learnable threshold. MePa_{pre} has comparable execution efficiency to MePa, but its accuracy is not as good as MePa. This aligns with our hypothesis as top-k pruning requires sorting after computing the similarity of all nodes, which greatly limits the efficiency of model execution. Manually set predefined threshold may not be optimal, thus greatly impacting model performance. Our proposed learnable threshold pruning combines the benefits of top-k pruning and predefined threshold pruning by adaptively learning the optimal threshold, achieving the optimal trade-off between accuracy and efficiency.

Effect of Pruning at Different Granularities: MePa integrates multiple granularity pruning: channel pruning, node pruning, and edge pruning. To explore the effect of each pruning operation, we explored three variants that use one pruning technique at a time in MePa: MePa_{CP} denotes only utilizing channel pruning, MePa_{EP} denotes only utilizing edge pruning, and MePa_{NP} denotes only utilizing node pruning. Table VII summarizes the experimental results of GCN on three graph datasets, including model accuracy and speedup (compared to HyGCN). We can conclude with the following observations: (1) Only employing one single pruning technique can still reduce the inference cost. Compared to edge pruning and node pruning, channel pruning can yield more performance gains. (2) MePa pruned at multiple granularities to obtain better efficiency than the variants that employ one single pruning.

VII. CONCLUSION

Existing typical GNNs leverage the neighborhood aggregation scheme to subtly aggregate the feature information from neighbor nodes to update the node representation. However, this complex computational model of GNNs greatly limits its deployment on resource-constrained edge devices. In this paper, we propose an algorithm and hardware co-design of GNN accelerator, referred to as MePa. By in-depth analysis of GNN limitations, we propose an efficient message-passing algorithm that dynamically prunes task-irrelevant feature channels/edges/nodes during execution. In this way, the overall complexity of GNN is dramatically reduced with negligible accuracy loss. Furthermore, we devise a novel architecture to support the dynamic pruning algorithm and translate it into performance improvements. Elaborate pipelines and specialized optimizations combine to improve performance and reduce power consumption.

REFERENCES

[1] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.

[2] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 249–270, Jan. 2022.

[3] M. Tzelepi and A. Tefas, "Graph embedded convolutional neural networks in human crowd detection for drone flight safety," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 5, no. 2, pp. 191–204, Apr. 2021.

[4] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 974–983.

[5] L. Yao, C. Mao, and Y. Luo, "Graph convolutional networks for text classification," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 7370–7377.

[6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–14.

[7] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 4438–4445.

[8] J. Sun, W. Zheng, Q. Zhang, and Z. Xu, "Graph neural network encoding for community detection in attribute networks," *IEEE Trans. Cybern.*, vol. 52, no. 8, pp. 7791–7804, Aug. 2022.

[9] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, vol. 31, pp. 5165–5175.

[10] L. Ma et al., "Neugraph: Parallel deep neural network computation on large graphs," in *Proc. Annu. Tech. Conf.*, 2019, pp. 443–458.

[11] M. Yan et al., "HyGCN: A GCN accelerator with hybrid architecture," in *Proc. IEEE Int. Symp. High Perform. Comput. Architecture*, 2020, pp. 15–29.

[12] S. Liang et al., "EnGN: A high-throughput and energy-efficient accelerator for large graph neural networks," *IEEE Trans. Comput.*, vol. 70, no. 9, pp. 1511–1525, Sep. 2021.

[13] S. Abadal, A. Jain, R. Guirado, J. López-Alonso, and E. Alarcón, "Computing graph neural networks: A survey from algorithms to accelerators," *ACM Comput. Surv.*, vol. 54, no. 9, pp. 1–38, 2021.

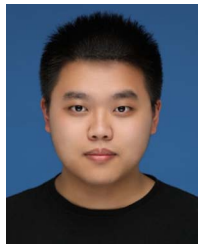
[14] T. Geng et al., "AWB-GCN: A graph convolutional network accelerator with runtime workload rebalancing," in *Proc. 53 rd Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2020, pp. 922–936.

[15] Y. Rong, W. Huang, T. Xu, and J. Huang, "DropEdge: Towards deep graph convolutional networks on node classification," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–17.

[16] A. I. Arka, B. K. Joardar, J. R. Doppa, P. P. Pande, and K. Chakrabarty, "DARe: Droplayer-aware manycore ReRAM architecture for training graph neural networks," in *Proc. IEEE/ACM Int. Conf. On Comput. Aided Des.*, 2021, pp. 1–9.

[17] C. Zheng et al., "Robust graph representation learning via neural sparsification," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 11458–11468.

[18] D. Luo et al., "Learning to drop: Robust graph neural network via topological denoising," in *Proc. 14th ACM Int. Conf. Web Search Data Mining*, 2021, pp. 779–787.
[19] Z. Gu, J. Li, and L. Chen, "Are all edges necessary? a unified framework for graph purification," 2022, *arXiv:2211.05184*.
[20] T. Chen, Y. Sui, X. Chen, A. Zhang, and Z. Wang, "A unified lottery ticket hypothesis for graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 1695–1706.
[21] P. W. Battaglia et al., "Relational inductive biases, deep learning, and graph networks," 2018, *arXiv:1806.01261*.
[22] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
[23] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1024–1034.
[24] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–17.
[25] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI Mag.*, vol. 29, no. 3, pp. 93–93, 2008.
[26] A. Katharopoulos and F. Fleuret, "Not all samples are created equal: Deep learning with importance sampling," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 2525–2534.
[27] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 3538–3545.
[28] M. Liu et al., "Towards deeper graph neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 338–348.
[29] M. Lin et al., "HRank: Filter pruning using high-rank feature map," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1529–1538.
[30] J. Chang, Y. Lu, P. Xue, Y. Xu, and Z. Wei, "Automatic channel pruning via clustering and swarm intelligence optimization for CNN," *Appl. Intell.*, 2021, pp. 17751–17771.
[31] Y. Xie, S. Li, C. Yang, R.-W. Wong, and J. Han, "When do GNNs work: Understanding and improving neighborhood aggregation," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, 2020, vol. 2020, pp. 1303–1309.
[32] X. Liu et al., "Survey on graph neural network acceleration: An algorithmic perspective," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 5521–5529.
[33] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.
[34] A. Bojchevski et al., "Scaling graph neural networks with approximate pagerank," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 2464–2473.
[35] S. A. Tailor, J. Fernandez-Marques, and N. D. Lane, "Degree-quant: Quantization-aware training for graph neural networks," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–22. [Online]. Available: https://openreview.net/forum?id=NSBrFgJAHg
[36] J. Chen, T. Ma, and C. Xiao, "FastGCN: Fast learning with graph convolutional networks via importance sampling," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–15.
[37] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 257–266.
[38] J. Li, A. Louri, A. Karanth, and R. Bunescu, "GCNAX: A flexible and energy-efficient accelerator for graph convolutional neural networks," in *Proc. IEEE Int. Symp. High- Perform. Comput. Architecture*, 2021, pp. 775–788.
[39] C. Chen, K. Li, Y. Li, and X. Zou, "ReGNN: A redundancy-eliminated graph neural networks accelerator," in *Proc. IEEE Int. Symp. High- Perform. Comput. Architecture*, 2022, pp. 429–443.
[40] H. Gao and S. Ji, "Graph u-nets," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2083–2092.
[41] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient sparse attention architecture with cascade token and head pruning," in *Proc. IEEE Int. Symp. High- Perform. Comput. Architecture*, 2021, pp. 97–110.
[42] C. Chen, K. Li, X. Zou, and Y. Li, "DyGNN: Algorithm and architecture support of dynamic pruning for graph neural networks," in *Proc. 58th ACM/IEEE Des. Automat. Conf.*, 2021, pp. 1201–1206.
[43] N. P. Jouppi et al., "In-Datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.
[44] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," 2019, *arXiv:1903.02428*.
[45] S. Thoziyoor, N. Muralimanohar, J. Ahn, and N. P. Jouppi, "Cacti 5.1," Tech. Rep. HPL-2008-20, HP Labs, 2008, pp. 1–37.
[46] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible DRAM simulator," *IEEE Comput. Architecture Lett.*, vol. 15, no. 1, pp. 45–49, Jan.–Jun. 2016.



Xiaofeng Zou received the Ph.D. degree in computer science and technology, Hunan University, Changsha, China, in 2023. His research interests include data mining, machine learning, and deep learning.

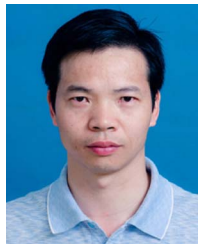
Cen Chen (Senior Member, IEEE) received the Ph.D. degree in computer science from Hunan University, Changsha, China. He is currently a Professor with the School of Future Technology, South China University of Technology, Guangzhou, China. He has authored or coauthored several research articles in international conferences and journals on machine learning algorithms and parallel computing, such as MICRO, HPCA, DAC, IEEE TRANSACTIONS ON COMPUTERS PEER-REVIEWED JOURNAL, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, AAAI, ICDM, ICPP, ICDCS, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, and IEEE TRANSACTIONS ON CYBERNETICS. His research interests include parallel and distributed computing, computer architecture, machine learning, and deep learning. He is also an Associate Editor for the IEEE TRANSACTIONS ON COMPUTERS.

Luochuan Zhang received the bachelor's degree in robotic engineering from the South China University of Technology, Guangzhou, China, in 2023. His research interests include machine learning and deep learning.

Shengyang Li is currently working toward the bachelor's degree with the School of Future Technology, South China University of Technology, Guangzhou, China. His research interests include machine learning and deep learning

Joey Tianyi Zhou (Senior Member, IEEE) received the Ph.D. degree in computer science from Nanyang Technological University, Singapore, in 2015. He is currently a Scientist with the Institute of High Performance Computing, Research Agency for Science, Technology, and Research, Singapore. Dr. Zhou was the recipient of the Best Poster Honorable Mention at ACML 2012, the Best Paper Nomination at ECCV 2016, and the NIPS 2017 Best Reviewer Award.

Wei Wei (Senior Member, IEEE) received the Ph.D.degree from the Xi'an Jiaotong University, Xi'an, China, in 2010. He is currently an Associate Professor with the School of Computer Science and Engineering, Xi'an University of Technology in China, Xi'an. He has more than 100 papers published or accepted by international conferences and journals. His research interests include mobile computing, image processing, and distributed computing.



Kenli Li (Senior Member, IEEE) received the Ph.D. degree in computer science from the Huazhong University of Science and Technology, Wuhan, China, in 2003. His major research interests include high performance computing, parallel computing, grid and cloud computing. He has authored or coauthored more than 200 research papers in international conferences and journals such as IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, and DAC.